

CHALLENGE · SPRINT 03

GoodWeAI

EV ChargeOps Assistant para condomínios residenciais

FIAP x GoodWe Brasil · Prompt and Artificial Intelligence · Ciência da Computação · Turma 1CCPK · 2026.2

DESAFIO	SOLUÇÃO ENTREGUE
A operação compartilhada de eletropostos exige consulta simples de consumo, faturas, incidentes, políticas e dados de moradores, mantendo controle de acesso.	Chatbot local com perfis de síndico e morador, interface Streamlit, PostgreSQL, RAG em ChromaDB e LLaMA 3.2 via Ollama.

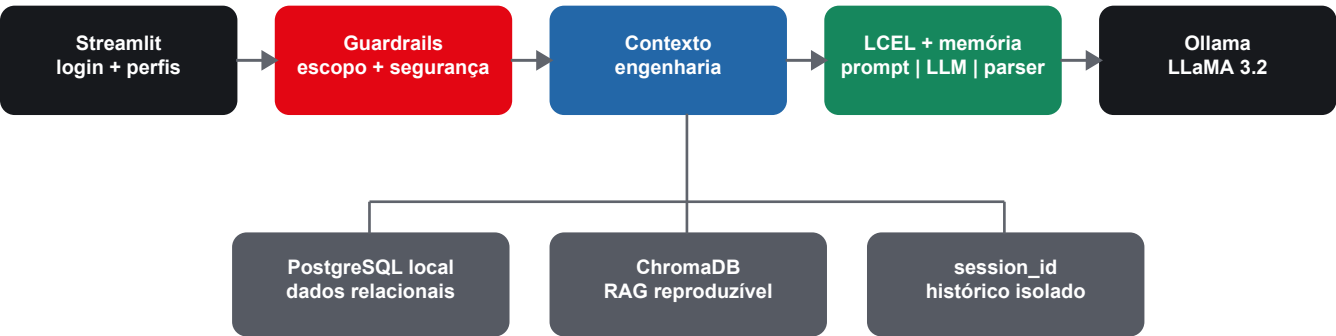
Equipe e responsabilidades

Integrante	RM	Responsabilidade principal
Rodger Costa Rios	571438	Coordenação, arquitetura e fluxo LCEL
Felipe Pereira Restivo	570712	Interface e experiência no Streamlit
Gabriel Rodrigues Zappelloni	572060	RAG, ChromaDB e base de conhecimento
Kenichi Caio Yamamoto	569815	PostgreSQL, modelos e persistência
Maykon de Lima Silva	574022	Guardrails, evals, modelos e documentação

Resultado da evolução

- Núcleo conversacional reestruturado em LangChain LCEL.
- Memória isolada por sessão, saída estruturada Pydantic v2 e context engineering.
- RAG reproduzível, guardrails determinísticos e suíte de avaliação automatizada.
- Solução inteiramente local, sem envio dos dados do condomínio a APIs externas.

1. Arquitetura atual



O Streamlit concentra login, cadastro, chat e painéis por perfil. Antes da inferência, a entrada passa pelos guardrails. O contexto combina PostgreSQL local, documentos do RAG e histórico da sessão. A LCEL envia o prompt ao LLaMA 3.2 e registra a resposta.

Evolução da Sprint 2 para a Sprint 3

Aspecto	Sprint 2	Sprint 3
Interface	HTMLs separados	Aplicação Streamlit por perfil
Orquestração	Fluxo acoplado	LCEL: prompt ChatOllama parser
Contexto	Prompt + consultas	Fontes separadas, priorizadas e delimitadas
Conversação	Histórico persistido	Memória por session_id e limite de tokens
Qualidade	Testes manuais	Guardrails + evals + Pydantic
RAG	ChromaDB local	Manifesto e reconstrução reproduzível

Stack e responsabilidade

Componente	Papel
PostgreSQL local	Usuários, perfis, recargas, faturas, incidentes e histórico
ChromaDB	Embeddings e recuperação semântica dos documentos
Ollama + LLaMA 3.2	Inferência local sem custo por token
LangChain LCEL	Composição declarativa e testável do pipeline
Streamlit	Experiência unificada para moradores, síndicos e administradores

2. Evolução técnica da Sprint 3

LCEL	MEMÓRIA POR SESSÃO
O pipeline usa ChatPromptTemplate ChatOllama StrOutputParser. Entradas do banco, RAG e pergunta permanecem explícitas e testáveis.	RunnableWithMessageHistory isola conversas por session_id. ConversationTokenBufferMemory limita o contexto a 2.000 tokens.

Saída estruturada com Pydantic v2

A classe ConsultaRecarga valida veículo, potência em kW, conector, local e dúvida. No caso de teste 'BYD Dolphin, 7,4 kW, residencial', o modelo retornou os três campos esperados e respeitou os limites definidos pelo schema.

Context engineering

- Prompts específicos para síndico/administrador e morador.
- Blocos separados para regras, dados relacionais, documentos e histórico.
- Prioridade explícita: PostgreSQL → RAG → memória; ausência de dados gera resposta de limitação.
- Conteúdo recuperado é tratado como dado, não como instrução, mitigando injeção indireta.

RAG reproduzível

Elemento	Implementação
Fonte	5 documentos Markdown em knowledge_base/
Manifesto	IDs, títulos, arquivos, coleções e hashes SHA-256
Coleções	4 documentos em atendimento; 5 em síndico
Reconstrução	preparar_rag.cmd / preparar_rag.py
Proteção	Somente documentos marcados como base_reproduzível são substituídos

Pergunta → Guardrails → PostgreSQL + RAG + Memória → LCEL → Ollama → Resposta

3. Segurança e avaliação



Guardrails antes do RAG e do LLM

Categoria	Comportamento esperado
Prompt injection / jailbreak	Recusa revelar ou substituir instruções internas
Fora do escopo	Limita o atendimento a mobilidade elétrica e GoodWe
Segurança elétrica	Não orienta intervenção; indica profissional habilitado
Jurídico e financeiro	Não produz parecer nem promessa de retorno
Produto não verificado	Não inventa especificação ausente na base

Comparação histórica de modelos

Taxa de sucesso nos 10 casos



Modelo	Aprovados	Latência média	Tokens de saída	Decisão
LLaMA 3.2:3b	10/10	0,904 s	709	Selecionado
Qwen3:4b	8/10	2,537 s	1.250	Não selecionado

Os dois modelos foram submetidos ao mesmo conjunto de dez casos. O LLaMA aprovou todos e foi aproximadamente 2,81 vezes mais rápido naquele ambiente. Os resultados históricos foram preservados; a execução integrada mais recente também obteve 10/10 com LLaMA.

4. Execução, evidências e conclusão

Execução local reproduzível

```
1. instalar.cmd
2. Criar o banco PostgreSQL local cargaai e configurar o .env
3. python setup_banco.py
4. python setup_ev.py
5. python atualizar_prompts_sprint3.py
6. preparar_rag.cmd
7. ollama pull llama3.2:3b
8. executar.cmd
```

Evidências no repositório

Evidência	Local
LCEL, memória e Pydantic	services/llm_service.py e models/consulta_recarga.py
Guardrails	services/guardrails/
RAG reproduzível	knowledge_base/, preparar_rag.py e services/rag_service.py
Casos e resultados	evals/eval_set.json e evals/sprint3_results.json
Comparação de modelos	docs/relatorio_modelos.md
Interface	streamlit_app.py

Limitações conhecidas

- A execução depende de PostgreSQL e Ollama instalados no computador da demonstração.
- A base RAG incluída é acadêmica e deve ser substituída por políticas reais do condomínio.
- Latência e consumo de tokens variam conforme hardware, GPU e estado de carregamento do modelo.

Conclusão

A Sprint 3 transformou a solução da Sprint 2 em um núcleo conversacional modular, testável e seguro. A arquitetura local preserva privacidade e elimina custos de API; LCEL, memória, structured output, RAG reproduzível e guardrails fornecem rastreabilidade para a demonstração. A suíte automatizada confirmou 23 testes unitários e 10 evals aprovados.

Repositório: github.com/rodgercr/ev-chargeops-assistant
Entrega: 21/09/2026 · FIAP · Turma 1CCPK